IoT Access Control

Mid-Quarter Progress Report

Steve Willoughby
Portland State University
Portland, Oregon, USA
slw3@pdx.edu

Jacob Olinger
Portland State University
Portland, Oregon, USA
jacoboli@pdx.edu

1 ABSTRACT

Internet of Things (IoT) systems include a multitude of sensors which "publish" data streams to another multitude of "subscribers". Between them sits a "broker" which routes these published messages to all the subscribers interested in them. However, there is a need for more robust access control to govern who can subscribe to potentially sensitive data, and under what conditions they may do so. Complicating things further, even an authorized subscriber may only have a valid need to access sensor data at certain times, in certain locations or under certain circumstances.

2 INTRODUCTION

Much of our increasingly-automated infrastructure relies on small, internet-connected devices, or the so-called "Industrial Internet of Things" (IIoT).[1] Among the numerous protocols used to enable these technologies to function is the MQ Telemetry Transport protocol (MQTT). This is a simple, lightweight plaintext message passing protocol which is designed to put a low overhead on small, low-power devices. As a result, it does not by default support encryption, authentication, message integrity validation, or access controls. It is an example of a "publisher/subscriber" system (commonly "pub/sub" for short) where sensor devices collect readings and other telemetry data as "topic publishers", sending them to a central broker service. This, in turn, retransmits the telemetry data to client systems, other servers, or even other IoT devices such as device controllers which "subscribe" to that data. Further, other independent which are in control of, e.g., automated building systems, publish messages to devices to adjust building airflow or activate door locks or whatever other device tasks they are in charge of, all through MQTT messages moving through the system.

3 RELATED WORK

I have a bunch of content to paste in here from my thesis work…

4 PROJECT DESCRIPTION

Our project seeks to add fine-grained, high-performance access control to MQTT pub/sub message broker systems for IoT networks.

It differs from the current state of the art in that most current offerings only go as far as basic ACL rulesets.

[1] As this class project is, with instructor approval, a portion of the research work toward Steve's Ph.D. dissertation, the text in the introduction section and other areas in this report is quoted from his research work in progress on this topic without further explicit reference, as this is considered all part of the same research work being done, and the results of this project will eventually be incorporated into the published papers and dissertation work as well.

5 TIMELINE

Our original estimated timeline was:

- **15 Apr** Proposal Presentation.
- **19 Apr** Project scope and plan finalized following feedback.
- **26 Apr** Design for experimental simulated IoT network.
- **03 May** Implementation of IoT clients and brokers.
- **10 May** Control runs completed, Mid-Quarter Report.
- **17 May** Add access controls to broker, 2nd experiments.
- **24 May** Additional experiments and scale tests.
- **31 May** Complete data analysis.
- **05 Jun** Final Report ready.

5.1 Progress to Date

As suggested by the color-coded dates above, we are about a week behind schedule due to the time required to search for suitable IoT client programs to use for publishers and subscribers and for the time to fully settle on the access control policy model to be used in the broker modifications we will be employing.

We have obtained the clients and have an installed broker instance, and are just about ready to begin the control runs now.

6 RESULTS AND ANALYSIS

We will set up simulated traffic and measure the performance impact of our new controls on the base system

We will run a number of test cases to demonstrate that the access controls are effective at limiting access to only authorized subscribers who meet the policy restrictions within the given domains.

6.1 Experimental Setup

We are working in a compartmentalized environment, which allows us to develop the server (MQTT broker) and clients (simulated publishing and subscribing devices) in relative isolation. The clients are in Docker containers, allowing them to be instantiated identically as many times as needed in the experimental environment, each running inside its isolated container but able to send and receive network traffic with the server.

This setup allows each team member to separately develop the server and client components away from the actual test environment as well, allowing development and debugging in a way that does not interfere with running experiments.

The server used to perform all the experimental runs, and upon which our performance benchmarks is based, was an Intel® Core™ i9-10980XE (36 cores) running at 4.80 GHz configured with 128 Gb of memory. It was running the Arch-based Garuda Linux distribution

6.13.7 Zen Linux kernel at the time of the experimental results reported here.

7 CONCLUSIONS

8 INDIVIDUAL CONTRIBUTIONS

The efforts were divided between Steve and Jacob along the boundaries of broker and client systems, with Steve handling the broker, including the setup of the Mochi MQTT broker and making the access control enforcement modifications to it, and Jacob setting up the containerized instances of the MQTT client programs acting as publishers and subscribers as well as the orchestration of those clients' actions in sending their traffic as part of our experimental runs.

9 REFERENCES